

Reflog recovery

You reset too far, checked out the wrong branch, or rebased and thought a commit was gone

Reflog before panic

- Run reflog to see recent moves: commits, checkouts, resets
- Each line has an index like HEAD at one, meaning one step ago
- If a hard reset dropped your tip
- Reflog is local only and entries expire after a while, so recover soon

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Reflog

INTERACTIVE TOOL

Reflog recovery

“Lose” a commit with `reset --hard`, then find it in `git reflog` and bring it back with `reset` or a recovery branch.

Starter example: main tip is mul commit c4d10aa. Hard-reset to drop it, then recover via `reflog (HEAD@{1})` or a recovery branch.

Load starter example

Challenges 0 / 22 cleared

Quiz: **what:** Reflog records where? Answer: HEAD pointed

Answer

Show hintCheckNextIdle

Quiz: whatQuiz: localQuiz: @{0}Starter tipHard resetReflog remembersSelect HEAD@{1}Recover resetRecover branchQuiz: listQuiz: undo resetDangling shownQuiz: expireCommit growsDouble hardQuiz: vs logBranch keeps tipQuiz: rebaseHEAD@{1} afterRoundtripQuiz: shareStarter depth

Idea

Reflog

Local diary of where HEAD pointed. Survives “lost” commits after `reset/rebase`.

Not on remote

Reflog is per-clone and expires. Push recovery branches if teammates need the commit.

Starter tipAccident: hard reset

Reflog recovery lab starter

learn_git — git reflog

Real Git practice

```
wsl - module13-git-reflog/examples/reflog

# Track A - real Unix
# real shell session (Track A)
# practice repo: /tmp/git-rf-49uAx1

$ git init

$ echo "v1" > notes.md

$ git add notes.md

$ git commit -m "First"

$ git branch -M main

$ echo "important" >> notes.md

$ git add notes.md

$ git commit -m "Important work"

$ git reflog
c7d7b37 HEAD@{0}: commit: Important work
75d74b8 HEAD@{1}: Branch: renamed refs/heads/master to refs/heads/main
75d74b8 HEAD@{3}: commit (initial): First

$ git reset --hard HEAD~1
HEAD is now at 75d74b8 First

$ git log --oneline
75d74b8 First

$ git reflog
75d74b8 HEAD@{0}: reset: moving to HEAD~1
c7d7b37 HEAD@{1}: commit: Important work
75d74b8 HEAD@{2}: Branch: renamed refs/heads/master to refs/heads/main
75d74b8 HEAD@{4}: commit (initial): First

$ git reset --hard HEAD@{1}
HEAD is now at c7d7b37 Important work

$ git log --oneline
c7d7b37 Important work
75d74b8 First
```

Real shell — reset, reflog, recover

Real Git practice — try these

```
# git init — create a new repository here  
git init
```

```
# echo ... > notes.md — first commit  
echo "v1" > notes.md
```

```
git add notes.md
```

```
git commit -m "First"
```

```
git branch -M main
```

```
# echo ... >> notes.md — commit you might “lose”  
echo "important" >> notes.md
```

```
git add notes.md
```

Pitfalls to watch

- Reflog does not travel to the remote, recovery is for your local clone
- Entries expire; do not treat reflog as permanent backup
- Prefer reset hard to HEAD at one only when you are sure that entry is the state you want
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter and recover a dropped commit from reflog
- On real Git, reset back one commit, then restore with HEAD at one
- When you are ready